Deep Green AI: Energy Efficiency of Deep Learning across Language–Framework Ecosystems

Leonardo Pampaloni[1*], Marco Pagliocca[1*], Enrico Vicario[1] and Roberto Verdecchia[1]

[1]University of Florence, Florence, Italy.

*Corresponding author(s). E-mail(s): leonardo.pampaloni1@edu.unifi.it; marco.pagliocca@edu.unifi.it;
Contributing authors: enrico.vicario@unifi.it; roberto.verdecchia@unifi.it;

Abstract

Context. Does the language–framework ecosystem carry an energy price in deep learning? A fair answer needs every ecosystem to run the same experiment, measured the same way.
Objective. We quantify how ecosystem choice affects DL training and inference energy, and what the measurement apparatus must guarantee.
Method. ResNet-18 and VGG-16 are trained on three datasets at $\mathbf{32 \times 32}$ in 7 ecosystems (Python with PyTorch, TensorFlow and JAX; C++ with LibTorch; Java with Deeplearning4j; R with `torch`; Rust with `tch`), 5 runs per configuration. A specification enforced by 110 automated checks fixes the experiment. Hardware counters measure energy, with CodeCarbon as a second instrument. A contrast cell of 70 runs repeats both networks at $\mathbf{224 \times 224}$.
Results. Training energy differs across ecosystems by 7.4× to 9.8× per cell (median between-run CV 0.49 %); among the 3 stacks sharing one exported module and backend build it is 1.1×–1.6×. At $\mathbf{224 \times 224}$ the spread within the four LibTorch-lineage stacks compresses to 1.3×–2.8×, while across all 7 it widens to 15.3×–19.1×, carried by the one stack without a cuDNN path. CodeCarbon matches the counters on energy within 0.3 %, but its reported duration overruns the energy window in 75 % of blocks, understating derived power up to 13.0×.
Conclusions. Ecosystem choice materially affects DL energy, by a workload-dependent amount. The apparatus distorts the duration and power reported beside energy, not the energy. Specification, checker and raw records are public.

Keywords: Machine Learning, Deep Learning, Green AI, Energy Efficiency, Sustainability, Green IT

1 Introduction

The rapid advancement of Artificial Intelligence (AI), particularly Deep Learning (DL), has delivered progress across domains such as healthcare, natural language processing, environmental monitoring and autonomous systems, at a growing environmental cost: training and deploying state-of-the-art neural networks demands vast computational resources and energy (Strubell et al. 2019; Lacoste et al. 2019; Bender et al. 2021; Xu et al. 2021; Patterson et al. 2021; Luccioni and Hernandez-Garcia 2023; Kaack et al. 2022). Energy use and emissions vary widely with model architecture, dataset size, hardware infrastructure and even the geographic location of execution (Patterson et al. 2022; Luccioni et al. 2023); standardised benchmarks such as MLPerf Tiny have begun to incorporate energy measurements (Banbury et al. 2021); and sustainability has become a central dimension of AI research (Henderson et al. 2020; Schwartz et al. 2020).

The field has responded with *Green AI* (Schwartz et al. 2020; Verdecchia et al. 2023b), which promotes AI systems that are energy-conscious as well as accurate. Surveys report advances in lightweight architectures, efficient training and inference, model compression and hardware-aware optimisation (Xu et al. 2021; Mehlin et al. 2023; Tripp et al. 2024; Yarally et al. 2023), and evidence increasingly suggests that energy improvements need not degrade accuracy (Cruz et al. 2025).

Beyond algorithmic efficiency, research in software sustainability has shown that the choice of programming language itself can have a substantial impact on energy usage (Pereira et al. 2021; Georgiou et al. 2017; Pereira et al. 2017; Gordillo et al. 2024; Chandra et al. 2019; Mahadevappa and Figueira 2021; Maleki et al. 2017), and on the run time and memory that partly determine it (Nanz and Furia 2015). Nevertheless, while both Green AI and the environmental sustainability of programming languages have been independently investigated, their intersection remains largely unexplored. A first step was made by Georgiou et al. (2022), who analysed the energy footprint of Deep Learning frameworks within Python, and by Marini et al. (2025), who studied the impact of programming languages in AI workloads and state explicitly that they "purposely opted not to focus on GPU-intensive algorithms such as deep neural networks, leaving their consideration as future work". That is the gap this study addresses.

For a practitioner, however, a programming language is inseparable from its *ecosystem*: choosing a language for DL implies adopting a software stack of bindings, frameworks, runtimes and driver/toolchain dependencies. In real-world settings it is therefore more meaningful to evaluate *language–framework ecosystems* as they are adopted in practice than to isolate a "pure" language effect under synthetic conditions.

Prior work has compared energy across DL frameworks *within* one language (Georgiou et al. 2022; Alizadeh and Castor 2024), measured language-level energy for CPU-bound workloads (Pereira et al. 2021; Gordillo et al. 2024), and characterised the time, memory and accuracy, but not the energy, of cross-language DL bindings (Li et al. 2024). Energy across language-based DL stacks for GPU training and inference, where frameworks, engines, toolchains and dataset complexity all vary together (Weber et al. 2023), has not been measured.

This paper addresses this gap with an empirical study of *the impact of language–framework ecosystems on the energy efficiency of DL workloads*: two widely adopted convolutional neural networks (ResNet-18 and VGG-16) across 7 ecosystems, spanning five host languages (Python, C++, Java, R and Rust) and the frameworks each offers in practice (PyTorch, TensorFlow and JAX; LibTorch; Deeplearning4j; the `torch` and `tch` bindings), on three datasets (Fashion-MNIST, CIFAR-100 and Tiny ImageNet), with one declared contrast cell on Imagenette at 224×224 and batch 32 that raises per-sample compute.

A cross-ecosystem energy comparison, however, rests on two premises that are rarely stated and never visible in the resulting numbers: that every stack is running the same experiment, and that every stack is measured the same way. We found both far harder to satisfy than the literature suggests. The campaign behind an earlier version of this study (the audited campaign, Section 5), of exactly this shape, was executed carefully and produced an entirely coherent story; auditing it against its own source code afterwards showed that the stacks had drifted apart in learning rate, input shape, evaluation batching and data-loader parallelism, that the measurement tool had been configured five different ways across them, and that two stacks were capable of running the whole workload on the CPU while reporting nothing but a log line. None of this was visible in the energy figures. All of it moved them.

We therefore treat the apparatus as part of the contribution rather than as a preliminary: a written experiment specification enforced by 110 automated checks, so that a divergence fails a check instead of quietly changing a number; hardware energy counters (NVML for the accelerator, RAPL for the CPU) with CodeCarbon as a second reading over the same phase boundaries; and 5 independent, interleaved runs per configuration with per-epoch test accuracy, so that the run is the unit of analysis and energy can be normalised by the quality reached.

Our results are correspondingly more modest and more defensible than a bare ranking: training energy differs across ecosystems by 7.4× to 9.8× at between-run variation of order 0.49 %, accuracy converges only on the easiest dataset, the size of the spread depends on the workload, and the software estimator we ran beside the counters agrees with them on energy while misreporting the time it was spent over (Section 6).

The contributions of this work are as follows:

- A systematic empirical comparison of DL energy consumption across 7 language–framework ecosystems for GPU-based workloads, replicated 5 times per configuration and reported with between-run uncertainty and quality normalisation, together with a declared contrast cell that tests how the spread changes when a higher resolution raises per-sample compute;
- An experiment specification for cross-ecosystem energy studies, together with 110 executable conformance checks that verify it against the source, the built binaries and the campaign's own metric files, and a single measurement contract implemented by all 7 stacks;

- A dual-instrument characterisation of CodeCarbon against hardware energy counters over 12 600 measurement blocks, including a quantified failure mode of its reported duration that invalidates energy–time analyses built on it;
- A comprehensive replication package, including all data, scripts, and settings required to fully reproduce our results.[1]

Detail the argument does not need — the full specification, supplementary analyses for RQ1, RQ3 and RQ4, an illustrative industrial scenario, the complete defect catalogue and the extended threats to validity — is in Online Resource 1.

2 Background

Green AI (Schwartz et al. 2020; Verdecchia et al. 2023b; Cruz et al. 2025) promotes AI systems that are energy-efficient as well as accurate. Sustainability can be quantified as carbon footprint, financial cost or energy use (Kaack et al. 2022; Patterson et al. 2021); we focus on **energy consumption** in Joules, a direct and reproducible measure of computational effort that, unlike carbon footprint, does not depend on the energy mix and location of execution and so compares consistently across heterogeneous DL stacks under a uniform protocol.

We treat execution time as a secondary metric, not a proxy for energy. The prevailing position in the software-energy literature, built largely on CPU-bound benchmarks, is that lower runtime does not imply lower energy (Weber et al. 2023; Rua and Saraiva 2024; Pereira et al. 2021), and van Kempen et al. (2025) argue that much of the reported language effect is an artefact of how such comparisons are constructed. Our argument is adjacent: theirs concerns confounds in what is compared, ours a field of the instrument's output. We test the relation for a GPU-executed workload in Section 6.4, where the answer depends on which field of the instrument's output is read as the duration.

Deep Learning workloads divide into *training*, iterative parameter updates through backpropagation, and *inference*, forward passes on a trained model; their energy profiles differ and optimisations in one do not necessarily carry over to the other (Mehlin et al. 2023; Tripp et al. 2024), so we analyse them separately.

Language–framework ecosystems may influence energy through execution engines, runtime overheads, input pipelines, precision policies and the driver/toolchain. In GPU-based DL most computation is delegated to optimised backend kernels, so differences should be read at the *ecosystem level* rather than attributed to the language alone; some ecosystems add runtime layers, and this study does not attribute its measured spread to that mechanism (Section 6.2). Language choice, often treated as secondary, constrains the frameworks, runtimes and toolchains available, and with them the energy profile.

Energy measurement for workloads of this kind can be read at three places, and the choice of place matters more than the choice of tool. A *wall meter* sees the whole machine including power supply losses, fans and drives. *Hardware energy counters* —

[1] Replication package: https://github.com/Pampaj7/DeepGreen. The Zenodo record https://zenodo.org/records/17734884 archives the audited campaign and is superseded by this one.

NVML on NVIDIA accelerators, RAPL on the CPU package (AMD exposes it too, under Linux's `intel-rapl` names) — accumulate energy in the silicon and are read by software, not estimated by it. *Software estimators* such as `CodeCarbon` (Courty et al. 2026) sit on top of whatever the platform exposes and substitute a model wherever it exposes nothing.

Software estimators have become standard in sustainability-oriented AI research (Dodge et al. 2022; Patterson et al. 2022) and are the sole instrument in model compression analysis (Paula et al. 2025), optimiser efficiency benchmarks (Almog 2025), large-scale evaluations of Large Language Models (Luccioni et al. 2024) and cybersecurity systems (Aashish et al. 2026) — studies whose conclusions inherit whatever the estimator gets wrong. Run alongside hardware wattmeters, CodeCarbon tracks sensor readings closely under controlled conditions and deviates more on heterogeneous infrastructure (Aquino-Brítez et al. 2025; Bouza et al. 2023); validation against external ground truth across hundreds of AI experiments reports errors of up to 40%, and systematic underestimation of 20–30% in some settings (Fischer 2026).

That literature asks whether an estimator's number is close to the truth; we ask whether a study can tell when it is not. A correction factor fixes inaccuracy, but not an estimator that silently substitutes a model for a measurement, or pads a duration it reports as measured. We therefore instrument every measurement block *twice*: hardware counters as the primary instrument, and `CodeCarbon` started and stopped at the same phase boundaries as a second reading, so that agreement is an observable property of each block rather than an assumption. NVML plus RAPL is a *board-and-package* measurement: the accelerator card including its memory, plus the CPU package, excluding the power supply, fans, drives and host DRAM. We had no wall meter and report that boundary consistently rather than a mixture of measured and modelled terms that corresponds to no physical boundary at all; Section 6.5 reports what the two instruments say about each other.

3 Related Work

Online Resource 1, Table A.1 places this study against the nine studies closest to it on the two axes that matter here, how many language ecosystems are compared and how energy is measured: each of them stays within Python (Li et al. 2016; Georgiou et al. 2022; Alizadeh and Castor 2024), leaves deep learning out (Pereira et al. 2021; Gordillo et al. 2024; Chattaraj et al. 2025), measures no energy (Li et al. 2024), compares instruments rather than stacks (Jay et al. 2023), or relies on a software estimator in a single run (Marini et al. 2025). This study compares 7 stacks across five languages with NVML and RAPL counters, the estimator alongside, over 5 runs per configuration.

Two of these deserve more than a row. Alizadeh and Castor (2024) compare runtime infrastructures for the same models on the same GPU, and reach the ecosystem-level question from inside Python: their treatment is the runtime beneath one language, ours is the language–framework stack above it. Li et al. (2024) compare TensorFlow and PyTorch bindings across five languages, which is our independent variable exactly, and measure run time, memory and accuracy rather than energy

— the gap this study fills, and the reason we treat the measurement boundary as a contribution rather than a preliminary.

Jay et al. (2023) compare software power meters, CodeCarbon among them, against physical meters on CPU and GPU, and establish that a measurement window must comfortably exceed a *sampling* meter's interval; on our platform the estimator reads accumulating registers instead (Section 6.5). What we add is orthogonal: a characterisation of the *duration* an estimator reports beside its energy, which to our knowledge has not been examined.

4 Research Methodology

This work adopts an empirical methodology grounded in the principles of software engineering empirical experimentation (Feldt and Magazinius 2010; Ampatzoglou et al. 2019; Sjøberg and Bergersen 2023; Verdecchia et al. 2023a). Our objective is not to propose a new algorithm or framework, but to systematically analyse how choosing among language–framework ecosystems (including their supported runtime and toolchain) can impact the energy efficiency of DL workloads. To this end, we designed an *in vitro* controlled experiment that treats ecosystems as independent variables, DL workloads as experimental objects, and energy consumption as the primary dependent variable (Basili et al. 1994; Cook and Campbell 1979; Campbell and Stanley 1963).

Research goal and questions. Following the Goal-Question-Metric (GQM) paradigm (Basili et al. 1994), our goal is formulated as:

> *Analyse* DL training and inference,
> *for the purpose of* quantifying and comparing their energy consumption,
> *with respect to* the choice of DL ecosystems (i.e. language bindings, frameworks/libraries, supported runtimes/execution engines, and associated toolchains),
> *from the viewpoint of* software engineering researchers and practitioners,
> *in the context of* computer vision image classification workloads.

Research questions. We ask four questions:

- **RQ1:** *How does language–framework ecosystem choice affect the energy efficiency of DL* training*?*
- **RQ2:** *How does language–framework ecosystem choice affect the energy efficiency of DL* inference*?*
- **RQ3:** *How do energy efficiency and execution time relate across different DL ecosystems?*
- **RQ4:** *To what extent are the answers to RQ1–RQ3 determined by the measurement apparatus rather than by the ecosystems?*

RQ4 is not a robustness check appended to the other three: a cross-ecosystem comparison is an inference from an instrument to a claim about software, so we treat the apparatus as an object of measurement in its own right.

Objects of study. We study two canonical convolutional architectures, ResNet-18 (He et al. 2016) and VGG-16 (Simonyan and Zisserman 2015), chosen for their

distinct computational characteristics (residual connections vs. sequential depth) and above all because nearly every DL framework ships them *off-the-shelf*, so that our measurements capture each framework's own implementation rather than a reimplementation of ours (Xu et al. 2021; Mehlin et al. 2023; Tripp et al. 2024).

Three datasets vary workload complexity: *Fashion-MNIST*, a lightweight baseline and drop-in replacement for MNIST[2] with 28×28 grayscale images in 10 classes, 60 000 for training and 10 000 for test (Xiao et al. 2017); *CIFAR-100*, a canonical benchmark of 32×32 colour images in 100 classes, 50 000 and 10 000 (Krizhevsky 2009); and *Tiny ImageNet*, a proxy for ImageNet-scale tasks with 64×64 colour images in 200 classes, 100 000 and 10 000 (Le and Yang 2015).

All datasets are resized once, offline, to a uniform 32 × 32 (`scripts/normalise_dataset_resolution.py`), so each stack's own resize is a no-op and no framework's resampling filter can enter the comparison. Before the change, 4 of the 7 stacks decoded measurably different pixels on Tiny ImageNet — the one dataset that has to be downsampled — with means that agreed and variances that did not. A common resolution preserves the datasets' differences in colour, class count and size while removing input resolution as a confound (Marini et al. 2025; Georgiou et al. 2022), keeps the classification layer fixed across datasets, reflects how canonical models are reused in practice (Cruz et al. 2025; Verdecchia et al. 2023b), and suits frameworks such as `tch` and Deeplearning4j that are more reliable at standardised sizes (Pereira et al. 2021; Gordillo et al. 2024). It may interact with framework-specific memory and kernel selection, and it keeps per-sample compute small, so one declared contrast cell at 224 × 224 (S7, Section 6.6) tests what changes when that compute rises.

Independent and dependent variables. Our primary independent variable is the *DL ecosystem*, i.e. the language–framework stack as adopted in practice (including its officially supported runtime and CUDA/cuDNN toolchain). In our design, each ecosystem is treated as a single categorical treatment; we do not attempt to decompose effects into separate contributions of language, framework, runtime, or toolchain. The 7 ecosystems are listed in Table 1. A second independent variable is the *DL phase* (training and inference).

MATLAB with the Deep Learning Toolbox was part of the campaign behind an earlier version of this study (the audited campaign, Section 5) and is out of scope here: the toolbox does not expose the training loop at the granularity the specification of Section 4.1 requires, so its data pipeline, evaluation batching and measurement boundaries cannot be brought into conformance with the other 7.

Our dependent variables are the *energy consumption* of each measurement block, in Joules at a board-and-package boundary (accelerator card plus CPU package), and the *test accuracy* reached at the end of each epoch. Recording the second alongside the first is what allows energy to be normalised by achieved quality in Section 6.2; a fixed-budget comparison that does not record accuracy cannot distinguish an efficient stack from one that has simply learned less (Patterson et al. 2021; Dodge et al. 2022; Luccioni and Hernandez-Garcia 2023).

[2] https://github.com/zalandoresearch/fashion-mnist

Study design. We adopt a *black-box*, *fixed-budget* design: off-the-shelf implementations trained under identical hyperparameters for the same number of epochs (Pereira et al. 2017; Georgiou et al. 2017; Rua and Saraiva 2024), as practitioners use frameworks *in vivo*. Per-ecosystem tuning could improve each stack's absolute efficiency but would confound the ecosystem with the tuning; the price is that a fixed budget does not guarantee convergence equivalence, which is why accuracy is recorded. Framework-level execution engines and optimisations (graph compilation, runtime schedulers, backend kernels) are treated as *part of the ecosystem*, so observed differences are ecosystem-level effects, not isolated language-level behaviour.

Numerical precision policy. All 7 ecosystems run under one precision policy, and it is set rather than inherited: fp32 storage and arithmetic, with TF32 permitted on the accelerator's tensor cores for convolution and for matrix multiplication; mixed precision is enabled in no stack. A single variable carries the policy to every stack, and every run's manifest records the flags that resulted (Online Resource 1, Sect. B.1). It belongs to the specification because of what happened when it did not (Section 6.2.2).

4.1 What "the same experiment" means, and how it is enforced

A fixed-budget comparison presumes that the budget is the same. Each ecosystem is written by a different person against a different API, and the places where stacks can silently diverge — a default learning rate, a data loader whose worker count follows the machine, an evaluation loop written one image at a time, an input left at its native resolution — are individually reasonable and invisible in the energy output. We therefore wrote the protocol down before running anything, as a specification with six parts and a seventh declaring the one contrast experiment, and made conformance machine-checkable. In brief (the full clauses are in Online Resource 1, Sect. B.2):

- **S1 — Model.** One canonical definition per architecture: one TorchScript module exported per (architecture, dataset) pair, its parameter count and parameter hash recorded in a manifest. The 3 stacks on the pinned LibTorch build (Python/PyTorch, C++, Rust) load that module; the others build the same architecture from the same initialiser and batch-normalisation convention, and all 7 agree on the sorted multiset of parameter-tensor shapes in all six cells. The three Python stacks, R and Java assert the expected parameter count at startup; C++ and the 32×32 campaign's Rust binaries receive it without checking it, a gap we found after the campaign and the saturation cell had run and report rather than patch.
- **S2 — Optimisation.** One optimiser, learning rate, batch size, epoch count, loss and seeding policy; the learning rate and batch size are checked against every file the campaign executes.
- **S3 — Pipeline.** $32 \times 32 \times 3$ inputs in raw $[0, 1]$, resized once offline; a bounded and equal number of loader workers; batched evaluation over the whole 10 000-image test split; and a per-run fingerprint of the decoded split over its 30 720 000 pixel values, so the pipelines are shown to agree rather than asserted to.

- **S4 — Backend.** One LibTorch build across the 3 stacks that share it and the one precision policy above; the accelerator is asserted at startup and, for Rust, against the built binary with `readelf -d`.
- **S5 — Measurement.** One measurement bridge, sampling interval, tracking mode, set of counters and unit, and one per-epoch record — training loss, test loss and test accuracy — persisted by every stack and checked against the campaign's own metric files.
- **S6 — Replication.** 5 independent, interleaved runs per configuration, each seed threaded into data order and initialisation.
- **S7 — Accelerator-saturation cell.** One declared contrast cell, never pooled with the campaign: the same 7 stacks, instruments and 5 repetitions, two ten-class architectures, Adam at 1×10^{-4} for 30 epochs at batch 32, on Imagenette resized once offline to 224×224 (Section 5).

110 automated checks enforce the specification against the source of every ecosystem, the configuration of every tracker, the built binaries and the campaign's own metric files, and fail the build when a stack drifts; 110 pass and 0 fail. A specification that lives only in a paper is a description of intent; expressed as executable checks, it fails at the moment of the divergence rather than at peer review. Building the suite meant first discovering that several checks could not fail, and the campaign then found what no static check could: a stack that failed on its first training step with every check green (Online Resource 1, Sect. B.3). Static enforcement is a floor, not a ceiling.

Residual, documented divergences. Three could not be removed (Online Resource 1, Sect. B.4). Deeplearning4j 1.0.0-M2.1, the last release of its API, links the CUDA 11.6 runtime and resolves its convolutions without cuDNN: it is the only stack of 7 that reaches no cuDNN kernel, and a conformance check now watches that property. It cannot be fixed, and we have one reference point for what it may cost: disabling cuDNN in PyTorch, holding hardware, model and data constant, costs 13.07× the energy and 16.42× the time on ResNet-18 at batch 128. That is PyTorch's own non-cuDNN fallback, not nd4j's im2col path, so it neither estimates nor bounds Java's penalty; it shows only that a missing convolution backend can cost a great deal, so Java's cost here may in part reflect that backend rather than a language. And R's `torch` binding cannot switch a script module between training and evaluation mode, so R builds the architecture instead of loading the shared module; its shapes are verified against the other six.

4.2 Measurement procedure

Energy is read from hardware counters at the boundaries of each measurement block: NVML's `nvmlDeviceGetTotalEnergyConsumption`, an accumulating millijoule register on the accelerator, and the RAPL package-energy counters exposed at `/sys/class/powercap` for the CPU, with wraparound at `max_energy_range_uj` handled explicitly. The reported quantity is their sum, identical for every ecosystem. Host DRAM, of order ten watts, is in neither term, since this platform exposes no `dram` RAPL domain (Online Resource 1, Sect. B.5). `CodeCarbon` (Courty et al. 2026) runs

over the same window as a second reading, in machine tracking mode at a one-second sampling interval and the default `api_call_interval` of 8, configured identically for all 7 stacks through a single shared bridge. We name that last parameter because Section 6.4 shows it is the one that decides whether a reported duration is a duration.

A *measurement block* is one phase of one epoch: 12 600 of them in total, each carrying two energy readings, a duration, and the test accuracy reached. We reserve the word for that unit. The six (model, dataset) combinations we report results over are *cells*, and the twelve (model, dataset, phase) groups the statistical tests run over are *test groups*. Because tracking is machine-wide, no other work runs on the host during a campaign; blocks measured while a background download was in progress during development were discarded and the protocol amended rather than corrected after the fact.

Replication and the unit of analysis. Each of the 42 configurations is executed 5 times as an independent process with a distinct seed, giving 210 runs. 206 of them ran *interleaved*, so that drift in machine state is spread across conditions instead of aliasing onto whichever ecosystem happened to run last; the remaining 4, all JAX on VGG-16, were stopped at their first batch by a defect and replayed 2 days later (Section 9). All uncertainty we report is between-run: the 30 epochs within a run share an initialisation, a JIT outcome, an allocator state and a thermal trajectory, and treating them as repeated measurements would understate uncertainty.

Quality normalisation. We report energy per epoch, per unit of accuracy achieved, and to first reach a target accuracy fixed per dataset — the last being closest to a practitioner's question, and the one a fixed-budget design cannot answer on its own.

Reproducibility and rigor. Two of the three ways a stack can ignore the seed it is handed are checked against its source. Full determinism cannot be guaranteed across stacks with non-deterministic GPU kernels, which is one reason the design replicates. The replication package (Section 1) holds the source of all implementations, run scripts, environment specifications and raw data (Cruz et al. 2025; Verdecchia et al. 2023b; Feldt and Magazinius 2010; Sjøberg and Bergersen 2023). How seeding failed in the audited campaign is recorded in Online Resource 1, Sect. B.5.

5 Experimental Execution

Experimental setup. All experiments reported here were executed on a single dedicated workstation: an NVIDIA GeForce RTX 3090 (Ampere, 24 GB GDDR6X, 350 W board power limit, driver 595.84), an AMD Ryzen 9 5900X (12 cores / 24 threads), 30 GB of RAM and NVMe storage, running Ubuntu 26.04 LTS (kernel 7.0.0). The machine was otherwise idle for the duration of the campaign, which whole-machine energy tracking requires (Online Resource 1, Sect. C.1).

Three campaigns of this design appear in the paper, under fixed names. The *audited campaign* is the one behind an earlier version of this study; it ran on a dual-socket Intel Xeon Gold 5418Y server with an NVIDIA L40S, absolute energy figures are not comparable between the two machines, and no comparison in this paper is made across them. The *superseded campaign* is the first execution of the present design

Table 1: Ecosystems, frameworks, versions and backends, as executed. The 3 stacks marked † share one pinned LibTorch build and load one exported TorchScript module: the internal control group. R (‡) links a LibTorch of its own and builds the same architecture — same lineage, different backend, different artefact. Precision policy and parameter counts are common to all stacks (Section 4.1).

Ecosystem	Framework / Library	Version	Backend
Python	PyTorch / TorchVision†	v2.7.0 / v0.22.0	LibTorch 2.7.0, CUDA 12.8
	TensorFlow / `tf_keras`	v2.19.1 / v2.19.0	cuDNN 9.3.0, CUDA 12 wheels
	JAX / Flax / Optax	v0.10.2 / v0.12.8 / v0.2.8	cuDNN 9.24.0, CUDA 12 wheels
C++	LibTorch†	v2.7.0	LibTorch 2.7.0, CUDA 12.8
Java	Deeplearning4j	v1.0.0-M2.1	ND4J CUDA 11.6, cuDNN unused
R	`torch`‡	v0.17.0	bundled LibTorch, CUDA 12
Rust	`tch`†	v0.20.0	LibTorch 2.7.0, CUDA 12.8
Instrumentation			
	CodeCarbon	v3.3.0	identical config, all stacks
	NVML / RAPL counters	driver 595.84	read by the shared harness

on this workstation, before the fixes of Sections 6.2.1 and 6.2.2. The *reported campaign* is its re-execution, on which every result here rests; "the campaign" without a qualifier means this one. The saturation cell (S7) is reported apart from all three.

Implementations and toolchains. The 3 stacks of the control group (Table 1) are pinned to one LibTorch build (2.7.0, CUDA 12.8) and load one exported module per (architecture, dataset) pair (S1). Four ecosystems cannot join them: TensorFlow and JAX bring their own backends; Deeplearning4j 1.0.0-M2.1 links CUDA 11.6, has no cuDNN path and no later release, and now carries its own redistributables; and R's `torch` 0.17.0 bundles its own LibTorch. The Python stacks live in three separate virtual environments, whose CUDA wheels conflict. A Julia implementation (`Flux`, `Lux`) was abandoned in a pilot over unresolved gradient-update problems (Online Resource 1, Sect. C.2).

Procedure. Every ecosystem trains for 30 epochs at batch 128 with Adam at a learning rate of 1×10^{-4} and cross-entropy loss, on 32×32 inputs in raw $[0, 1]$ (grayscale expanded to three channels), with 2 loader workers and evaluation batched over the whole test split, a configuration chosen to reflect canonical practice while keeping the implementations comparable (Xu et al. 2021; Mehlin et al. 2023; Banbury et al. 2021; Pereira et al. 2021); Table 2 lists it in full.

The saturation cell. Specification S7 adds one contrast cell, executed after the campaign on the same host: Imagenette[3], ten ImageNet classes, 9 469 training and 3 925 test images, resized once offline by the shorter side to 224 and centre-cropped to 224×224. Batch size is 32 in both phases, because VGG-16 at 224×224 does not fit the card's 24 GB at batch 128, and the classifier has ten outputs. The cell is 7 ecosystems $\times$ 2 architectures $\times$ 5 repetitions, 70 runs of 30 epochs, interleaved, at the

[3] https://github.com/fastai/imagenette

Table 2: Hyperparameter configuration, identical across all ecosystems; how each row is verified is in Online Resource 1, Sect. C.6.

Parameter	Value
Number of epochs	30
Batch size	128
Optimiser	Adam
Learning rate	1×10^{-4}
Loss function	Cross-entropy
Input resolution	32×32, resized once offline; no stack resizes
Input channels	3 (grayscale datasets expanded)
Numerical precision	fp32 storage, TF32 allowed, no mixed precision; one campaign-wide setting
Accelerator	asserted at startup, CPU fallback refused; the Rust binaries checked for a linked CUDA library
Data loader workers	2, bounded and equal in every stack
Input normalisation	none; raw $[0, 1]$ inputs in every stack and every model
Evaluation	batched at the training batch size, over the whole 10 000-image test split
Seeding	one seed per run from the run contract, reaching data order and initialisation
Independent runs	5 per configuration, 206 of 210 interleaved

campaign's 2 loader workers, and lives in a campaign directory of its own; no figure in this paper aggregates across the two (Online Resource 1, Sect. C.3).

Measurement and provenance. Counter reads and `CodeCarbon` v3.3.0 (Courty et al. 2026) are started and stopped by the same code path, through a bridge whose synchronous acknowledgement matters: an earlier asynchronous version under-reported one stack's training energy by 24% and recorded its evaluation energy as zero (Online Resource 1, Sect. C.4). Every reported quantity is generated from the per-block records by the analysis pipeline (Online Resource 1, Sect. C.5).

6 Results

We report the campaign in the order the research questions were posed, after a subsection on repeatability that bounds everything after it. One departure is worth flagging: RQ4, on the apparatus, comes last but conditions everything before it, and a reader who wants to know how much to trust Sections 6.2–6.4 should read Section 6.5 first. Section 6.6 closes the results with the contrast cell of specification S7, which asks how the answers to RQ1–RQ3 change when a higher resolution raises per-sample compute. Every interval, test and effect size is computed across the 5 independent runs of a configuration.

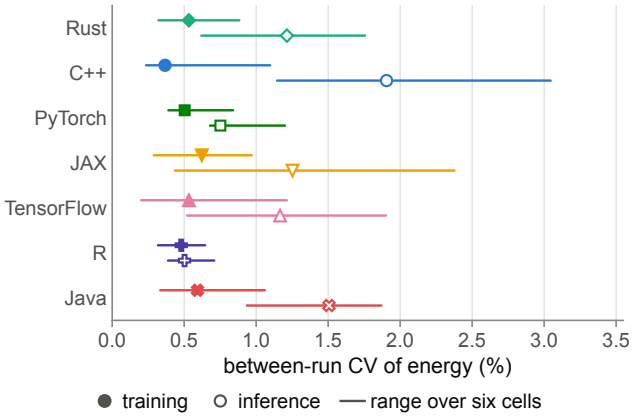


Fig. 1: Between-run coefficient of variation of energy, from 5 independent runs per configuration: the median over the six (model, dataset) cells (marker; filled for training, hollow for inference) and the range over them (line).

6.1 Repeatability of the measurements

How precisely the apparatus measures any one ecosystem bounds every comparison that follows. Across all 42 configurations, the median between-run coefficient of variation of training energy is 0.49 % and the worst is 1.21 %. Inference, whose blocks are several times shorter, is noisier but still tight: a median of 1.17 % and a worst case of 3.04 %. Figure 1 breaks this down by ecosystem.

Differences of a few percent between ecosystems are therefore resolvable, and those reported below are far larger. Low variance is a property of the measurement, not a guarantee about the thing measured.

6.2 RQ1: Impact of ecosystems on training energy efficiency

Figure 2 reports training energy per epoch for every ecosystem and cell, at a common board-and-package boundary; the means are tabulated in Online Resource 1, Table D.1.

The spread between the cheapest and the most expensive ecosystem ranges from 7.4× to 9.8× depending on the cell (Table 3), and which stacks occupy the two ends changes with the architecture: C++ is cheapest in every ResNet-18 cell, TensorFlow in two of the three VGG-16 cells and JAX in the third, while the most expensive is R on ResNet-18 and Java on VGG-16.

The shared-module control group. 3 ecosystems — Python/PyTorch, C++ and Rust — load the same exported module on the same pinned LibTorch build (S1). The group controls the model and the backend build, not everything else those three stacks do differently. Within it the spread is 1.1× to 1.6× (Table 4), against 7.4× to 9.8× across all 7. On ResNet-18 the group spans 1.3×–1.6× and accounts for at most 21 % of the log spread; on VGG-16 it spans 1.1×–1.2× and for as little as 3 %. On

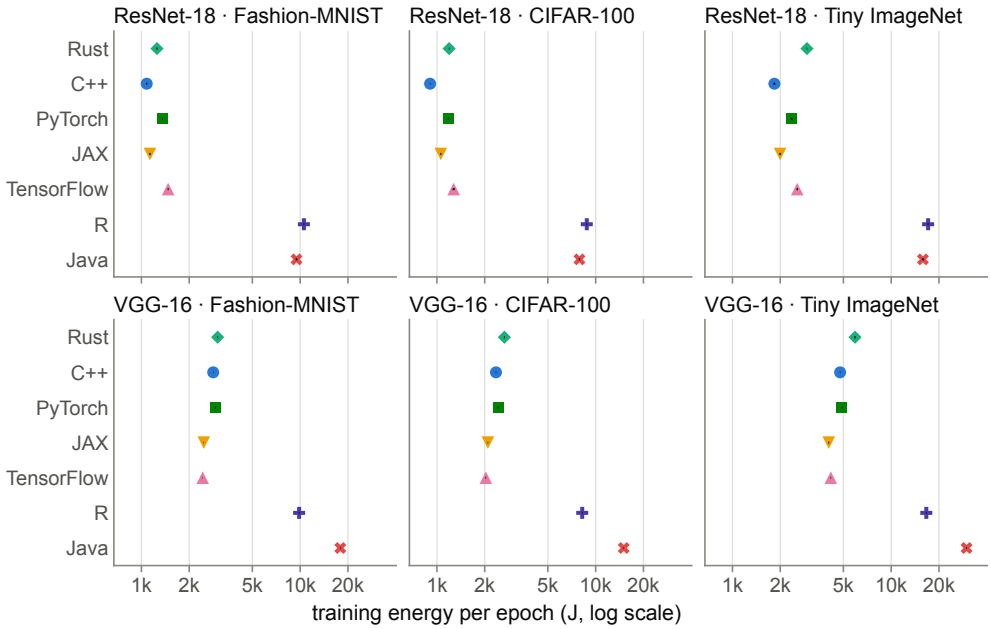


Fig. 2: Training energy per epoch, accelerator plus CPU package: mean over 5 runs (marker) and its 95% Student-t interval (black line; every interval is narrower than its marker). One logarithmic axis for all panels.

Table 3: Cheapest and most expensive ecosystem per cell, training energy per epoch.

Model	Dataset	Lowest		Highest	
			J		J Ratio
ResNet-18	Fashion-MNIST	C++	1 080	R	10 563 (9.8×)
ResNet-18	CIFAR-100	C++	906	R	8 792 (9.7×)
ResNet-18	Tiny ImageNet	C++	1 838	R	17 087 (9.3×)
VGG-16	Fashion-MNIST	TensorFlow	2 433	Java	17 910 (7.4×)
VGG-16	CIFAR-100	TensorFlow	2 033	Java	15 029 (7.4×)
VGG-16	Tiny ImageNet	JAX	4 044	Java	29 855 (7.4×)

both architectures, then, most of what separates ecosystems separates the stacks that do not share the module from the ones that do.

In the superseded campaign the same comparison read like binding overhead; it was a precision flag and unaligned initialisers, now fixed (Sections 6.2.1 and 6.2.2, Online Resource 1, Sect. D.2). Table 4 reports the spread that survives holding the model and the backend build fixed; a black-box design cannot decompose what is left, and we do not.

R is reported beside the control rather than inside it, because it shares neither the build nor the module (Section 4.1). Including it widens the "shared-module" spread

Table 4: Spread within the shared-module control group (one LibTorch build, module and data pipeline) against the full spread, per cell. The LibTorch-family column adds R.

Model	Dataset	All 7	Shared module & build (3)	LibTorch family (4)	Share of log spread
ResNet-18	Fashion-MNIST	9.8×	1.3×	9.8×	10%
ResNet-18	CIFAR-100	9.7×	1.3×	9.7×	12%
ResNet-18	Tiny ImageNet	9.3×	1.6×	9.3×	21%
VGG-16	Fashion-MNIST	7.4×	1.1×	3.5×	3%
VGG-16	CIFAR-100	7.4×	1.1×	3.5×	6%
VGG-16	Tiny ImageNet	7.4×	1.2×	3.5×	11%

Table 5: Final test accuracy (%) after the fixed epoch budget, mean over the 5 runs of each cell; *Spread* is the highest minus the lowest in a column. No run is excluded.

	Fashion-MNIST		CIFAR-100		Tiny ImageNet	
Ecosystem	R-18	VGG-16	R-18	VGG-16	R-18	VGG-16
Rust	90.02	92.60	28.88	33.74	14.01	15.09
C++	90.26	92.65	29.51	33.01	14.09	14.89
PyTorch	90.42	92.68	28.39	32.98	14.21	15.28
JAX	90.30	92.93	30.97	32.01	14.03	14.57
TensorFlow	90.31	92.74	28.23	33.90	13.92	14.52
R	90.45	92.51	34.73	34.65	17.22	14.68
Java	89.94	92.67	28.50	33.75	13.93	15.39
Spread	0.5	0.4	6.5	2.6	3.3	0.9

to 3.5×–9.8×, which would have been reported as binding overhead and is not: the difference between the two columns of Table 4 is the cost of one member that shares less than the label claims.

Energy at equal quality. A fixed-budget comparison rewards a stack for learning less, so Table 5 reports the final test accuracy each reached. On Fashion-MNIST all 7 stacks land within 0.5 percentage points of one another on either architecture, which is the strongest available evidence that they are running the same experiment — and a check the audited campaign could not perform, because no ecosystem wrote its accuracy to disk. We quote the per-architecture figure deliberately: pooling ResNet-18 and VGG-16, which reach different accuracies in opposite per-ecosystem directions, would report 0.3 points and would be measuring the pooling rather than the stacks. On the harder datasets the stacks separate more — to at most 6.5 points per cell, the widest cell of the same *Spread* row, with no run excluded because none collapsed (Section 6.2.1) — and there the energy comparison must be read alongside the accuracy reached rather than instead of it: on both harder datasets the stack that reaches the highest accuracy is also among the two most expensive to train (Figure 2).

Figure 3 plots the two together. Ranked by accuracy per kilojoule rather than by energy alone, C++ leads Java by 7.9× — a different quantity from the energy ratio,

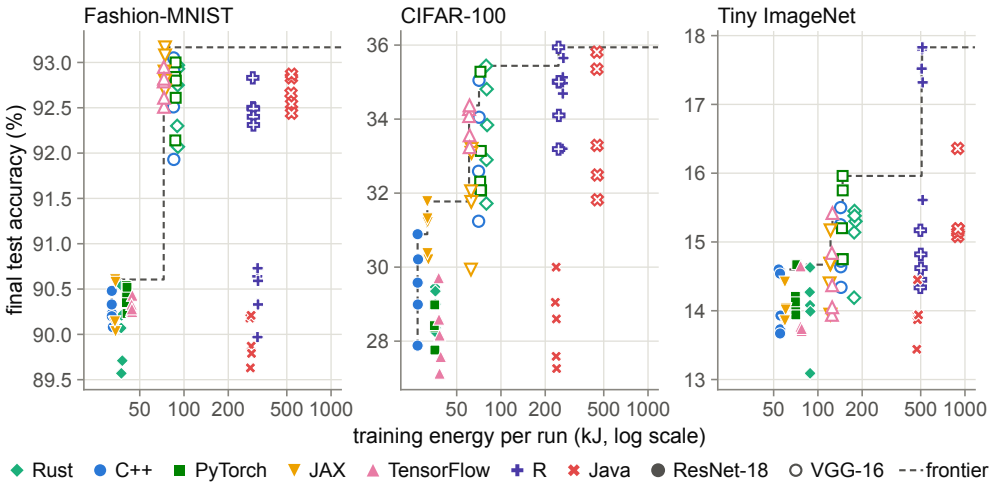


Fig. 3: Energy spent against accuracy reached, one point per run (filled: ResNet-18, hollow: VGG-16; one logarithmic energy axis). Dashed: the empirical frontier, left of which no configuration reaches the same accuracy.

and closer to the one a practitioner choosing a stack faces. Because accuracy per kilojoule differs by two orders of magnitude between datasets we give it per dataset as well: 8.0× on Fashion-MNIST, 8.0× on CIFAR-100 and 7.7× on Tiny ImageNet, with JAX, C++ and C++ best on each. The three agree with the pooled figure and with each other, which nothing guaranteed in advance.

Energy to a target. Table 6 gives the energy a stack spends before it first reaches a stated accuracy. On Fashion-MNIST every run of every ecosystem reaches 85 %, and the energy to get there spans 8.9× on ResNet-18, from C++ to R, and 6.3× on VGG-16, from C++ to Java — a like-for-like comparison at equal quality, made within one architecture. On CIFAR-100 every run reaches 30 % as well. On Tiny ImageNet 6 of the 7 stacks on ResNet-18, and 7 on VGG-16, never reach 20 % within the epoch budget; we show the empty cells, because "did not reach the target" is the result. The one stack that does reach it reaches it in every one of its runs, so its cell is a median over all 5 runs; with nothing to compare it against, it is a result about that stack rather than a comparison.

6.2.1 Runs that consumed the budget and learned nothing

In the superseded campaign 12 of 105 VGG-16 runs *collapsed*: on CIFAR-100 and Tiny ImageNet their loss sat at $\ln N$ for the N classes and their accuracy at chance for all 30 epochs, so they consumed 10.7 % of that campaign's training energy and learned nothing. ResNet-18 never does this in either campaign (0 of 105 runs here), and neither architecture does it on Fashion-MNIST. We had read the collapses as a property of the ecosystems. A controlled side experiment that varied only the initialiser — 0 collapses in 6 runs under He initialisation, and 2 and 4 under two variants of Glorot (Xavier) initialisation — showed that they were the weight initialiser, which

Table 6: Median training energy (kJ) to first reach a target accuracy, with the number of runs that reached it where that is not all 5; "never" marks a cell in which no run reached it. Each architecture is read on its own. Targets: 85 %, 30 %, 20 %.

Ecosystem	ResNet-18			VGG-16		
	F-MNIST *85%*	CIFAR-100 *30%*	Tiny-IN *20%*	F-MNIST *85%*	CIFAR-100 *30%*	Tiny-IN *20%*
Rust	1.3	6.0	never	6.0	26.7	never
C++	1.1	4.5	never	5.7	30.4	never
PyTorch	1.4	5.9	never	5.8	26.7	never
JAX	2.6	6.9	never	8.2	29.3	never
TensorFlow	2.8	7.4	never	5.8	21.0	never
R	10.2	26.0	86.4	19.2	82.3	never
Java	9.3	46.4	never	35.7	195.1	never

frameworks ship differently under one architecture name. Starting every stack from one canonical seeded export, with matched initialisers in the stacks that build their own layers, 0 of 105 VGG-16 runs collapsed here: an exact one-sided 95 % upper bound of 4.2 % on the susceptible cells, against 17 % measured there before, and 0.0 % of the 36.0 MJ spent here went on runs that learned nothing. A per-epoch discriminator separates collapse, where the training loss does not move, from a pipeline defect, where it falls while test accuracy stays at chance. It flagged 4 of the 5 runs of one configuration that carried a defect of our own — a Rust loader that trained on black images — and all 5 were deleted and the configuration re-executed (these are not the replayed JAX runs of Section 4.2). In the reported campaign it finds 0 pipeline defects and 0 collapses. With one run per configuration, nothing in an energy table distinguishes a collapsed run from an expensive one (Online Resource 1, Sect. D.3, with the superseded campaign's figure).

6.2.2 The precision policy, and what it was worth

The superseded campaign pinned TF32 off, with `cudnn.deterministic` on, in a harness through which only Python/PyTorch reached cuDNN. Of the other stacks, TensorFlow was handed a mixed-precision policy, a different axis, Java reaches no cuDNN kernel at all, and the rest took their framework's default, which on this card lets cuDNN use TF32. The reported campaign runs all 7 under one policy. The comparison rests on C++, Rust and R, whose policy did not change between the two. On CIFAR-100, the one cell in which nothing else changed, Python/PyTorch's ResNet-18 accelerator energy per training epoch is 3.14× higher with TF32 denied, while those 3 stacks move by at most 1.3 %; and the superseded campaign's Python/PyTorch-versus-C++ gap of 3.94×, which reads like binding overhead, is 1.25× in the reported one. A kernel-level probe on a resident batch puts the same flag pair at 4.04× the accelerator energy per step (TF32 denied alone, 3.62×; determinism alone, 0.99×). PyTorch also moved from a fresh torchvision model to the exported module between the two campaigns, so the campaign ratio bounds the precision effect from above. A precision policy was worth more here than any difference between bindings this study measures, and a comparison that does not state its policy is not comparing ecosystems (Online Resource 1, Sect. D.4, with the per-cell contrast table).

Table 7: Mean inference energy per evaluation pass (J), accelerator plus CPU package.

Ecosystem	ResNet-18			VGG-16		
	F-MNIST	CIFAR-100	Tiny-IN	F-MNIST	CIFAR-100	Tiny-IN
Rust	72	100	171	154	187	263
C++	58	72	74	111	113	115
PyTorch	173	181	184	239	238	276
JAX	154	165	166	262	262	268
TensorFlow	154	160	164	210	211	234
R	2 253	2 204	2 341	2 144	2 123	2 279
Java	1 094	1 081	1 076	2 593	2 600	2 713

RQ1 Findings: Impact of Ecosystems on Training Energy Efficiency

Measured at a common board-and-package boundary and replicated 5 times per configuration, training energy differs across ecosystems by 7.4× to 9.8× depending on architecture and dataset, with C++ cheapest in every ResNet-18 cell and the cheapest VGG-16 stack changing with the dataset. Between-run variability is small (median CV 0.49 %), so these differences are well resolved. On Fashion-MNIST final accuracy converges to within 0.5 percentage points on either architecture, so there the differences are energetic rather than qualitative. On the harder datasets the stacks do *not* reach the same accuracy — up to 6.5 points on a single (architecture, dataset) cell, with every run counted — and there the energy comparison must be read alongside the accuracy reached. Among the 3 stacks that load the same exported module on the same pinned backend build the spread is 1.1×–1.6× against 7.4×–9.8× across all 7, so most of the effect is between stacks that do not share a model and a backend. One cuDNN flag was worth more than any of it: denying TF32 to a single stack cost 3.14× its ResNet-18 accelerator training energy, which is why a cross-ecosystem comparison that does not state its precision policy is not comparing ecosystems.

6.3 RQ2: Impact of ecosystems on inference energy efficiency

Table 7 reports inference energy per pass over the test set, at the same boundary. Between the cheapest and the dearest ecosystem of a cell it spreads by 23.1× to 38.7×, against 7.4× to 9.8× in training.

This is also where the apparatus matters most. An inference pass takes well under a second in the fastest stacks, and blocks that short almost always carry seconds of tracker lifetime in the duration CodeCarbon reports (it is *padded*, Section 6.4), so power or time drawn from that field compares padding for exactly the fastest stacks. Our energies are *counter-bracketed*: read from the hardware counters at the two phase boundaries. That removes the padding but is not enough on its own, because a window measures only the work that finishes inside it: in the superseded campaign JAX's inference blocks closed over dispatched work that had not completed, and JAX was reported cheapest at inference. The blocks here force every asynchronous stack to completion before the window closes (Section 8). Inference is where such a study is most exposed to its instrument, and it is the phase practitioners deploy.

RQ2 Findings: Impact of Ecosystems on Inference Energy Efficiency

Inference energy spreads more widely across ecosystems than training energy (23.1×–38.7×), on blocks short enough that the instrument dominates any duration or power derived from the estimator's own fields; the energy rankings themselves agree between the two instruments (Section 6.5). Inference durations and power should be treated as claims about the apparatus until the window over which energy was accumulated is stated.

6.4 RQ3: Energy–time relationship across ecosystems

The claim that "faster is not greener" is a claim about two measurements, and it is only as good as the weaker of them. We therefore recompute it on counter-bracketed durations — the interval between the two counter reads that bracket a phase — rather than on any duration reported by the estimator.

On that basis, energy and time rank the configurations almost identically: Spearman $\rho = 0.96$ for training and 0.92 for inference, with 7.1 % and 11.5 % of configuration pairs discordant respectively. Those coefficients are pooled over all 42 configurations, so they are partly a statement about workload size: a bigger dataset costs both more time and more energy whatever the stack. Within a cell and phase — one architecture, one dataset, one phase, the 7 ecosystems — the same coefficient runs from 0.68 to 1.00 over the 12 test groups, so the relation is not an artefact of pooling workloads of different size. On CodeCarbon's reported durations, with everything else unchanged, the coefficients are 0.95 and 0.72, with 8.9 % and 22.0 % of pairs discordant: the reported duration weakens the inference relation and raises its discordant share from 11.5 % to 22.0 %, but the relation stays positive. Within this workload and platform, execution time is a good proxy for energy on either duration. What the reported duration does distort is power.

Figure 4a shows that the duration CodeCarbon reports is not the interval the energy was accumulated over. 25 % of blocks carry an excess of 0.01 s, that is none; 75 % carry seconds, around a median of 4.57 s; and 1 block sits at 13.38 s. These are the figure's 3 *modes*: clusters of the campaign's excess. The middle one spans 2.08 s and holds two sub-clusters a second or so apart, which the mode-finder does not separate. 75 % of blocks are therefore reported with a window seconds longer than the interval their energy was accumulated over.

The cause is a network call. `EmissionsTracker.stop()` resolves cloud metadata and then the host's geolocation over the network — two blocking HTTP requests — *after* the final energy reading, which is why only the duration is affected. Both results are cached on the tracker, and a background thread performs the same call once 8 measurements have accumulated, so a block pays for the lookups still outstanding when it ends. A probe on this host with no workload (Online Resource 1, Table E.1) measures the cost of `stop()` at three *levels*: 4.56 s with both lookups outstanding, 3.01 s with one, and 0.012 s for blocks longer than 12 s. Levels and modes are two views of one mechanism: the lowest mode is the no-lookup level, the middle mode holds both padded levels (its two sub-clusters), and the single block of the

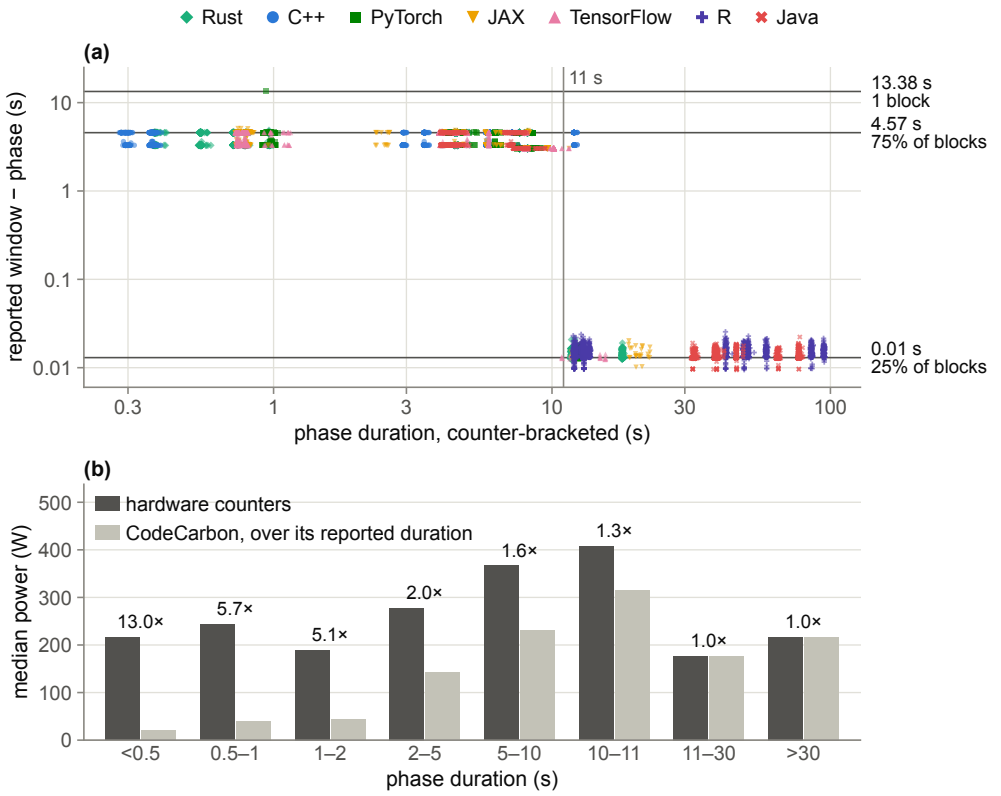


Fig. 4: (a) Per block, the reported duration minus the counter-bracketed phase, against the phase: 3 modes (horizontal lines, with median and share of blocks) and the threshold near 11 s (vertical line). (b) Median power per phase-length bin from the counters and from reported energy over reported duration, on the terms both measure, labelled with the median factor by which the second understates the first (Online Resource 1, Table E.3).

top mode matches no level. A single threshold at 11 s classifies 98.8 % of the 12 600 blocks correctly, and every inference block is under it, and padded, except R's (Online Resource 1, Sect. E). The padding is network latency on this host, not a constant of the tool, and an offline configuration or an `api_call_interval` of −1 removes it; a study that measures short phases and does not set that parameter records inflated durations, and nothing in the output says so.

The consequence for power needs no model at all: divide each instrument's own energy by its own duration (Figure 4b, Online Resource 1, Table E.3). Below half a second the estimator's fields give 20 W where the counters measure 216 W, a factor of 13.0. The distortion falls monotonically with block length and reaches unity above 11 s, where the reported window is the accumulation window and the two agree exactly. Both sides of that comparison are the terms the two instruments both measure; putting CodeCarbon's total in the numerator instead would mix this artefact

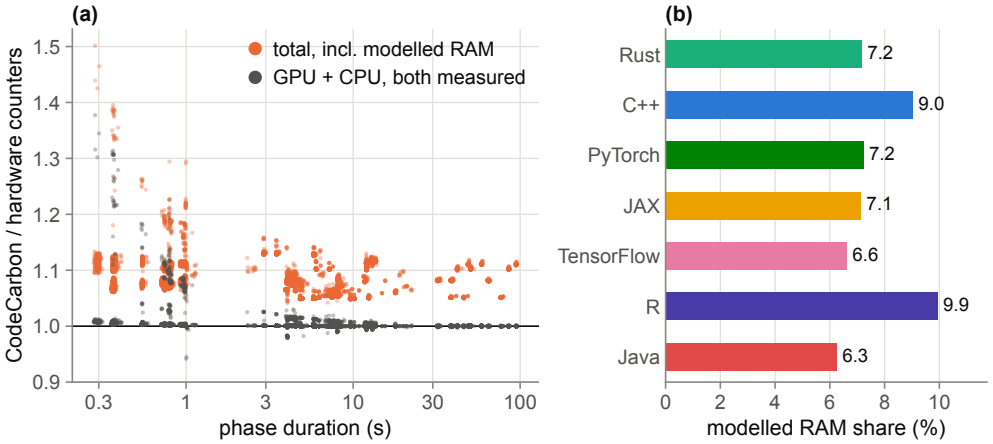


Fig. 5: (a) Ratio of CodeCarbon to the counters, per block, against phase duration. Grey: the GPU and CPU terms, which both instruments measure; they agree. Orange: CodeCarbon's total, which exceeds the counters by its modelled RAM term. (b) Mean modelled-RAM share of each ecosystem's reported total.

with the modelled RAM term criticised in Section 6.5. The shortest phase in the campaign takes 0.28 s and is filed as 4.86 s, a factor of 17.

The bias is monotone in phase length, so it falls hardest on the fastest ecosystems and on the inference phase, stretching their apparent time axis while leaving their energy untouched: that is how a study can obtain a misleading power comparison from correct energy measurements.

RQ3 Findings: Energy and Time

On counter-bracketed durations, energy and execution time rank the campaign's 42 configurations almost identically ($\rho = 0.96$ training, 0.92 inference): on this platform and workload, faster *is* greener. On the estimator's reported durations the relation weakens for inference ($\rho = 0.72$) but does not reverse, so those durations alone would not have produced the contrary finding here. What the estimator's duration field does distort is power: it pads 75 % of blocks with seconds of tracker lifetime that carry no energy, inflating the apparent runtime of the shortest block by 17× and understating derived power by up to 13.0×, most for the fastest stacks.

6.5 RQ4: How much of this is the apparatus?

Every block in the campaign carries two energy readings taken over the same phase boundaries. Figure 5 and Table 8 report what they say about each other.

On energy, the two readings agree — and it is important to say why. Over the whole campaign the ratio of CodeCarbon to counters is 1.0029 for the accelerator and 1.0053 for the CPU package, or 1.0034 for their sum, a mean per-block disagreement of 0.3 %; weighted by energy over the campaign it is 0.04 %.

Table 8: Agreement between the two readings over 12 600 blocks: the mean of per-block ratios (dominated by short blocks, which carry a fixed offset) and the ratio of campaign totals. The first three rows compare registers with themselves.

Quantity	Mean per block	5th–95th pct	Campaign-weighted
GPU, ratio (both read the NVML energy register)	1.003	1.000–1.006	1.000
CPU package, ratio (both read RAPL energy_uj)	1.005	1.000–1.022	1.001
GPU + CPU, ratio (the part both read)	1.003	1.000–1.008	1.000
CodeCarbon total over counters, ratio (incl. modelled RAM)	1.087	1.050–1.127	1.079
RAM share of the CodeCarbon total (per cent, not a ratio)	7.634	4.686–10.458	7.293

That is not an accuracy result. Where both counters are exposed, CodeCarbon reads *the same registers we do*: NVML's accumulated-energy register on the accelerator and the RAPL `energy_uj` files on the CPU package. The two are not independent instruments; they are two readers of one instrument. The evidence is in the residual itself. The accelerator terms differ by a median of 1.2 mJ — floating-point noise on the same integer register — and essentially all of the residual is the CPU term, a near-constant 0.48 J per block whose correlation with block duration is 0.02. That constant is the counter reads being nested immediately inside the tracker's own window rather than coinciding with it: at this machine's package power it corresponds to a window a few milliseconds wider, and it does not grow with the block.

This does not certify the estimator's accuracy against ground truth, which would need a wall meter and which independent validation work supplies (Fischer 2026). What it certifies is that where the counters are exposed the estimator's energy is not a model at all, so the failure modes worth worrying about lie where the counters are *not* exposed and the substitution is silent, and in the fields beside the energy. The resolution of the reference and the arithmetic of a fixed offset on short blocks bound the per-block comparison (Online Resource 1, Sect. F.1).

On what is not measured, they cannot agree. CodeCarbon's total exceeds the counters by 1.087×, and the excess is entirely its RAM term — 7.6 % of its reported total on average — which in machine tracking mode, the mode a whole-machine study needs, is derived from installed memory capacity rather than from memory activity: on this machine it infers four modules from 30.2 GB and yields a flat 20 W for every block, whatever the workload does to memory (Online Resource 1, Sect. F.2). No counter on this platform can confirm or refute it. The number is not wrong so much as it is not a measurement, added silently to a figure otherwise composed of measurements, and on a host with more installed RAM it contributes more. Our own boundary has the opposite failing: the counters omit DRAM energy altogether. Neither is a whole-system number, and the difference between them is not an error bar.

The time between the phases belongs to the instrument. Between a run's first block and its last, tracked time ranges from 46 % (C++) to 100 % (R). The gaps are the instrument holding its window open: the reported-window excess correlates with them at $r = 0.98$ (largely a padded-versus-unpadded contrast) and accounts for 97 % of all untracked time, so coverage follows block length, not the ecosystem, and

per-phase energy is the right quantity to compare. The untracked time — 10.5 hours, at least 2.4 MJ or about 6 % of the reported energy at idle draw — is avoidable by the same parameter (Online Resource 1, Sect. F.3, with its Table F.1).

Does the instrument change the answer? We recomputed the ecosystem ranking of each test group under both readings. They agree in 9 of 12 test groups — 3 of 6 in training and 6 of 6 in inference — and the disagreements are adjacent-pair swaps, so no ecosystem moves more than one rank. Every disagreement is a VGG-16 training group; the inference groups, whose sub-second blocks this paper treats as the fragile ones, agree throughout. The substantive ordering of RQ1 is therefore unchanged, provided energy and not power derived from it is compared: ranked by derived power, the two readings disagree everywhere.

RQ4 Findings: The Apparatus

Over 12 600 doubly instrumented blocks, a widely used software estimator agrees with hardware energy counters to 0.3 % on the quantity both measure — because on this platform it is reading the same two registers, not estimating. That is the useful finding: where the counters are exposed, the estimator's energy is not the risk. The risks are the terms beside it. A modelled RAM term contributes 7.6 % of the reported total, which no counter can check. 75 % of blocks are reported with a duration seconds longer than the interval their energy was accumulated over, so power derived from those two fields is understated by up to 13.0×. Both produce plausible numbers; neither is visible from the estimator's output alone. The ecosystem ranking is nonetheless unchanged in 9 of 12 test groups — 6 of 6 at inference and 3 of 6 in training, where the disagreements are adjacent-pair swaps.

6.6 The accelerator-saturation cell

At 32×32 per-sample compute is small. Over the training blocks 5 of the 7 stacks hold the accelerator at or above 85 % utilisation on VGG-16, within 14.9–93.8 %, but 1 on ResNet-18, within 5.5–85.1 %; R is the least loaded stack on both, and the median over the 14 (ecosystem, architecture) pairs is 64.3 %. Where the GPU is lightly loaded a study may be measuring host-side work — decode, collation, dispatch, binding — and calling it the energy cost of deep learning; if so, the ecosystem spread should compress when per-sample compute rises. We measured that in the one contrast cell the specification declares (S7): the same 7 stacks, architectures, harness and instruments, on Imagenette at 224×224 with batch 32, 70 runs. The cell changes resolution, batch size and dataset together, so it measures the joint effect of a heavier per-sample workload and not of accelerator loading alone. It is not pooled with the campaign; every quantity below is computed on one definition on both sides (Table 9, and in full Online Resource 1, Sect. G).

The cell loads the accelerator. On the 1 Hz `nvidia-smi` record over each run's training blocks, the median over the cell's (ecosystem, architecture) pairs is 89.2 %, from 18.1 % (R, ResNet-18) to 99.3 % (JAX, VGG-16), against 5.5 % (R) to 93.8 % (C++) at 32×32; per pair the median gain is 12.0 points. Two stacks fall well short of saturation: Rust on ResNet-18 does not rise at all, and R remains the least loaded stack on both architectures, at the specification's 2 loader workers, which we did not

Table 9: The accelerator-saturation cell (§S7) beside the campaign it contrasts. Training energy is the hardware counters over the counter-bracketed window – NVML board energy plus the RAPL CPU package domains – for one epoch: the median over the 5 independent runs of each cell, each run contributing its own per-epoch mean. Utilisation is the 1 Hz `nvidia-smi` record over each run's training blocks (the counter-bracketed interval of every training epoch, excluding start-up, evaluation and tracker overhead), on both sides; the record covers all 70 of the 70 runs at 224 and, having started after the main campaign did, 9–15 of the 15 runs of each ecosystem and architecture at 32×32. The last row of each panel is the ratio of the two spreads, 32×32 over 224: above 1× the ecosystem spread is narrower at 224×224, batch 32, on Imagenette than at 32×32. In training it narrows within the LibTorch lineage (2.64–3.48×) and widens across all seven ecosystems (0.39–0.64×). The cell changes resolution, batch size and dataset together, so neither ratio is attributed to any one of them.

Ecosystem	**kJ/epoch** 224	**s/epoch** 224	**util. (%)** 224	**util. (%)** 32×32
ResNet-18				
Rust	4.5	13.8	53.8	55.8
C++	3.3	8.2	89.3	85.1
PyTorch	3.5	9.3	77.0	54.3
JAX	2.6	6.6	92.4	31.9
TensorFlow	2.9	7.4	89.1	47.3
R	9.2	45.4	18.1	5.5
Java	39.5	120.9	87.6	40.3
Spread, 224	15.3×	18.4×	18.1–92.4	–
Spread, 32×32	9.3–9.8×	16.2–16.8×	–	5.5–85.1
Compression	0.61–0.64×	0.88–0.91×	–	–
VGG-16				
Rust	16.2	43.0	85.3	72.9
C++	15.0	37.6	97.8	93.8
PyTorch	15.3	38.8	94.5	88.8
JAX	12.0	30.7	99.3	89.3
TensorFlow	9.9	25.4	97.9	86.3
R	19.7	60.4	62.0	14.9
Java	189.3	499.2	99.2	88.1
Spread, 224	19.1×	19.6×	62.0–99.3	–
Spread, 32×32	7.4×	8.2–8.5×	–	14.9–93.8
Compression	0.39×	0.42–0.43×	–	–

In the energy and time columns a spread is the ratio of the dearest ecosystem to the cheapest, over the seven ecosystems of one architecture in the training phase, each ecosystem taken at its mean over runs as in Table 3 (the cells above are medians), and the 32×32 row gives the range of that ratio over the three datasets of the main campaign; in the utilisation columns, which are percentages, it is the range across the same seven. Compression is the 32×32 spread over the 224 spread. A cell reading '–' is a quantity that row does not define, or a cell no run of which the utilisation record covers.

vary. Every stack learned: the lowest single run of the cell (Java, ResNet-18) ends at 64.3 % test accuracy over ten classes.

Within the LibTorch lineage the spread compresses. The four stacks that descend from LibTorch — PyTorch, C++, Rust and R — spread by 9.3×–9.8× in ResNet-18 training over the three 32×32 datasets and by 2.8× at 224×224; on VGG-16, by 3.5× and 1.3× (Figure 6). Inference compresses further, and the accelerator-only spread at 224×224 is of the same size. The compression has one author: C++ is the cheapest member of the four and R the dearest for every architecture and phase at

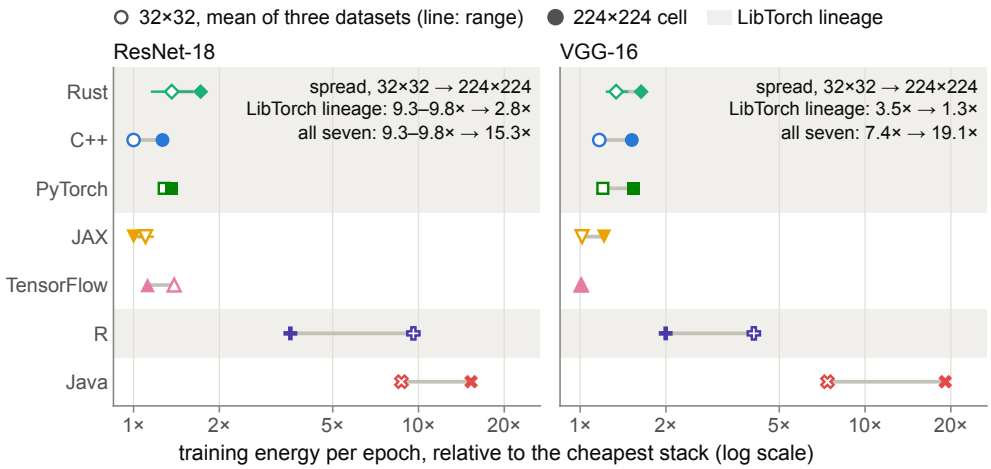


Fig. 6: Per-epoch training energy (mean over runs of each run's per-epoch counter mean, as in the spread rows of Table 9) as a multiple of the cheapest stack in the same regime: 32×32 (hollow; mean of the three datasets, each normalised to its own cheapest, range as a coloured line) and the 224×224 cell (filled). On the log axis distance is spread. Shaded rows: the LibTorch lineage.

both resolutions, so the family's spread is R's distance from C++, and that is the distance that closes — consistent with a host-side share in R's cost, which the cell does not decompose. The 3 stacks of the control group do not compress in training, where their spread was small to begin with.

Across all 7 the spread widens. Training energy spreads by 15.3× on ResNet-18 and 19.1× on VGG-16, against 9.3×–9.8× and 7.4× at 32×32; the accelerator-only spread is 15.1× and 19.0×, so the widening is not a host-side term. The dearest stack for every architecture and phase in the cell is Java. Setting Java aside, the other six spread by 3.6× and 2.0× in training, against 9.3×–9.8× and 4.0×–4.1× at 32×32: without Java the spread compresses, as it does within the LibTorch lineage; inference is mixed. Java is the stack that reaches no cuDNN kernel (Section 4.1), and at 224×224 its accelerator is busy on both architectures, so its cost in the cell is device work rather than host waiting — consistent with a convolution path without cuDNN being what separates it from the other six, but not a measurement of it: the 13.07× reference point was taken with PyTorch at 32×32, and we did not repeat it at 224×224.

The ordering partly survives. Over 12 comparisons of the 224×224 ordering with each 32×32 dataset's (exact permutation p), VGG-16 training keeps its order or nearly: $\rho = 1.00$, 1.00 and 0.96 against CIFAR-100, Fashion-MNIST and Tiny ImageNet. ResNet-18 training and inference on both architectures agree more weakly (ρ down to 0.54, p up to 0.2357). What moves is the top: C++, the cheapest ResNet-18 stack in training on every 32×32 dataset, is not the cheapest at 224×224, where it is JAX on ResNet-18 and TensorFlow on VGG-16, and in a single comparison up to 4 stacks move by two places or more. A ranking read off the campaign is a statement about its regime.

Faster is still greener. On the definition behind RQ3, run time and run energy rank the cell's 14 (ecosystem, architecture) pairs at $\rho = 0.93$ in training and 1.00 in inference, against 0.96 and 0.92 over the campaign's configurations; 5.5 % and 1.1 % of pairs are discordant. With the accelerator loaded, mean training board power sits between 149 W and 346 W across the cell on the same counter records, a range far narrower than that of the per-epoch times in Table 9, so time remains the larger factor in energy.

Saturation-cell Findings: Does the Spread Survive a Heavier Workload?

At 224 × 224 and batch 32, where median training-block utilisation is 89.2 %, the ecosystem spread does not collapse: across all 7 stacks training energy spreads by 15.3× (ResNet-18) and 19.1× (VGG-16), wider than at 32 × 32. Within the LibTorch lineage it compresses to 2.8× and 1.3×, because R, the stack that leaves the accelerator least loaded, closes most of its distance from C++; without Java the other six compress too, to 3.6× and 2.0×. The widening comes from Java, the one stack that reaches no cuDNN kernel and whose cost in the cell is device work; we did not bound the missing backend's share at this resolution. The VGG-16 training order is preserved, the cheapest stacks change, and faster is still greener ($\rho = 0.93$ training, 1.00 inference).

7 Discussion

7.1 Interpretation of Results

The ecosystem effect is large, and most of it lies outside the shared-module control group. That the language–framework ecosystem materially affects DL energy agrees with prior work (Georgiou et al. 2022; Marini et al. 2025; Pereira et al. 2021; Gordillo et al. 2024). The 3 stacks that share an exported module and a LibTorch build differ by only 1.1×–1.6× (Table 4), against 7.4×–9.8× across all 7. This is an observation, not an intervention: the other stacks differ in framework, execution engine and toolchain even where they share cuDNN, CUDA and the card, and a black-box design of this shape does not say which difference carries the spread. The absolute gap widens with dataset size and per-sample compute, so the choice matters most where energy does, and because runtime layers, pipelines, engines and toolchains all differ, these are ecosystem-level effects, not effects of a programming language.

The boundary is part of the result. How wide the factor is depends on where the measurement boundary is drawn, and the choice is invisible in the number. The audited campaign shows this on its own records, on other hardware and quoted for this point only: its ecosystem spread is 4.6× when energy is taken as the estimator's total and 8.5× when taken as its accelerator term alone. A constant host term added to every stack compresses ratios toward one, so any cross-ecosystem ratio is a statement about a boundary as much as about the stacks.

The phases rank ecosystems similarly but not identically. The same ecosystem is cheapest in both phases in 3 of 6 cells and the full ordering is identical in 0

of them, with Spearman correlations between each cell's training and inference energies of 0.54 to 0.93, so a choice made on one phase does not carry over to the other without its own measurement.

Faster *is* greener, on this platform and workload. This contradicts a well-established finding in cross-language energy work (Weber et al. 2023; Rua and Saraiva 2024; Pereira et al. 2021), within a scope: those studies largely concern CPU-bound benchmarks, where a language can trade instruction count against memory traffic and shift energy per second, while in our GPU-executed DL workload the accelerator held a narrow band of power, so time and energy were nearly proportional. It is a scope condition, not a refutation.

Quality must be measured, not assumed: an ecosystem that converges more slowly looks efficient because it accomplished less. Here every run trained, so no accuracy figure carries collapsed runs; in the superseded campaign they did (Online Resource 1, Sect. D.3).

7.2 Implications for Stakeholders

- **Researchers:** report energy alongside accuracy and the measurement boundary, and back a cross-stack study with a written specification, a mechanism that enforces it and a second instrument: most defects we found produced numbers of a plausible magnitude and ordering, and most were detectable only by such a mechanism (Section 8). We make the artefacts as important as the findings: a reader who doubts a figure can regenerate it from the replication package, and one who doubts the apparatus can run the checks.
- **Practitioners:** ecosystem choice has a first-order effect on energy, but must be weighed against maintainability, hiring, library availability and time to delivery: a gap of this size is not a mandate when the cheaper stack has a tenth of the ecosystem around it. A phase-specific split needs its own measurement. Online Resource 1, Sect. H translates the multipliers into hyperscale energy and cost, as a projection rather than a measurement.
- **Framework and tool designers:** the target we can name is the convolution backend: Deeplearning4j's final release reaches no cuDNN path, and one reference point shows what losing cuDNN can cost — PyTorch's own fallback took 13.07× the energy (Section 4.1). Measurement tools should report the interval over which energy was accumulated, and fail rather than substitute a model when a counter is unavailable.
- **Policy makers and funding agencies:** AI efficiency is multi-dimensional, and a single metric as a target invites Goodhart's Law. Reporting requirements should ask for the measurement boundary and the replication count: a figure without either is not auditable.

8 A catalogue of defect classes

Auditing the audited campaign, with its eight stacks, and repairing every stack against the specification turned up 34 defect *classes*: plausible mistakes in any study of this shape, several of them language or tool *defaults* rather than oversights, and most

invisible in the resulting measurements. Online Resource 1, Sect. I lists every class with its measured effect and whether a reader given only the released data could detect it. They concern the *instrument* (a unit mislabelled by 3.6×10^6, a padded duration, a modelled term in a measured total, device work outliving its window), the *implementation* (divergent learning rates, loaders, input shapes, initialisers and seeds; two of the audited campaign's stacks trained at ten times the stated learning rate, Online Resource 1, Table I.5), *placement* (a silent CPU fallback, a linker dropping the CUDA dependency) and *analysis* (collapsed and partial runs averaged in, epochs treated as replications). Only three classes are visible from the released data; the rest require reading the code, a second instrument, or a quality metric the study thought to record. 12 of the 34 are ours: most were caught by a mechanism rather than by attention; two escaped every mechanism and surfaced only as symptoms — an accelerator drawing idle power, and the campaign failing on its first batch with every check green. Enforcement beats discipline, and our own discipline was not sufficient either.

9 Threats to Validity

We group threats conventionally, quantify them where the data allow, and give the full discussion in Online Resource 1, Sect. J.

The reported campaign replaces one of our own. The superseded campaign was confounded by a precision policy in force for one stack only, four networks under the name VGG-16 and diverging initialisers; every ecosystem comparison here is measured after aligning them and re-executing all 210 runs. What carries over is the apparatus — the instrument comparison, the reported-window result, the boundary argument and the source-level defects — each re-derived from this campaign.

Internal validity. Whole-machine tracking means that concurrent work contaminates a measurement: the host was otherwise idle, and blocks measured during development while a download ran were discarded and the protocol amended. Toolchain heterogeneity (CUDA 11.6 to 12.8) cannot be removed, since an ecosystem is partly defined by its toolchain; the control group bounds it without saying what the surviving spread is. Implementation drift is attacked by the specification and its checks, with the residual gaps stated in Section 4.1; two of our own defects escaped every check, so we claim only that a divergence now has to survive one, and that the campaign is the last check in the sequence.

The defect catalogue names the lowest mean accelerator power among the stacks as the signature of a stack silently running on the host, and we applied that diagnostic to our own data. R draws the lowest accelerator power, 127 W against up to 258 W for the others on a 350 W card. The CPU term clears it: its package power, 53 W, is inside the 50–68 W band of every other stack, and the S4 assertion refuses a CPU device. It uses the accelerator at a low duty cycle — 4.5–5.0 % utilisation on ResNet-18 and 12.1–13.4 % on VGG-16 over each run's wall clock — and its epochs are the longest in the campaign. Placement is verified for Rust too, whose binaries run on the CPU by default because `--as-needed` drops the CUDA dependency (S4).

Construct validity. Our construct is energy at a stated boundary, not "sustainability": accelerator board plus CPU package, excluding the power supply, fans, drives, chipset and host DRAM. We had no wall meter, and a board-and-package number must not be compared with a wall-meter number from another study. The counters are not exempt from scrutiny: RAPL's package energy is itself modelled on some parts (Khan et al. 2018), and NVML rests on internally averaged board telemetry, so for the shortest blocks both instruments are near the edge of what they resolve — a joint property of the pair, which does not bear on the reported-duration finding. Two counter-side defects of ours were fixed for this campaign: a RAPL wrap guard with the wrong units, inert for every block but never needed (the longest block is 95.7 s), and manifests without machine state, which every run now records. Every figure is un-baselined: on an idle host this machine draws 64 W at the boundary (25 W at the accelerator, 39 W at the CPU package), 23 % of the mean power of a measured block. The CPU term is mostly the machine being on — its idle draw is 68 % of its campaign mean, against 11 % for the accelerator — yet contributes 21 % of reported energy; this boundary carries no measured host-side term, which is one reason we do not apportion the spread between host-side and accelerator work.

External validity. We studied two convolutional architectures on three image classification datasets at 32×32, and one contrast cell at 224×224; results may not generalise to other model families, to transformers, or to other resolutions and batch sizes, where the balance between accelerator and host-side work differs; the contrast cell, itself one point, shows the stacks moving by different amounts as that balance moves, so the campaign's multipliers are not constants to carry elsewhere. The platform is a single-accelerator desktop, whose host share of energy is smaller than a multi-socket server's, and MATLAB's absence is a conformance limit, not a result about MATLAB.

Conclusion validity. Epochs within a run are not independent, so every interval and test here is computed on 5 independent runs per configuration, 206 of the 210 runs interleaved. Five repetitions is a small sample for a rank test, and we report what that costs. A Kruskal–Wallis test rejects equality of ecosystems in all 12 test groups ($p \leq 1.2 \times 10^{-5}$, $\varepsilon^2 \geq 0.95$), and 99 % of the 252 pairwise comparisons carry a large Cliff's delta — an effect size saturated by a between-run coefficient of variation under half a per cent, which records that groups separate, not by how much. Yet the number of those pairs significant after Holm correction is 0, and it cannot be otherwise: the smallest exact two-sided Mann–Whitney p for five runs against five, 0.0079, becomes 0.167 over 21 comparisons. The design shows that the ecosystems differ but cannot order a particular pair, so ordering claims rest on effect sizes and intervals. The superseded campaign's collapse rates warranted an exact Freeman–Halton test ($p = 0.0040$) rather than the χ^2 and permutation tests we first ran; on the reported campaign it returns $p = 1.0000$, because no ecosystem collapsed a run. Finally, 4 runs (3 configurations, all JAX) were replayed 2 days after the rest and are not interleaved. Re-executing one completed configuration (PyTorch, ResNet-18 on Fashion-MNIST) in a third window bounds the drift: training energy is 1.2 % lower, 1.6 within-window standard deviations, and comparable; inference is 3.2 % lower, 4.0 standard deviations (5.6 J per block), which we carry as a between-window uncertainty

of that order on the late inference figures — small against the ecosystem differences but not noise, and obtained from one configuration in a different ecosystem.

10 Conclusion

This paper presented an empirical study of how *language–framework ecosystems* affect the energy efficiency of *Deep Learning* workloads, across 7 ecosystems, two convolutional architectures and three datasets, replicated 5 times per configuration and instrumented twice per measurement block. Four findings stand out.

First, ecosystem selection substantially shifts operational efficiency: training the same model on the same data costs between 7.4× and 9.8× more energy on one ecosystem than another, at between-run variability of order 0.49 %. Among the 3 stacks that share one exported module and backend build the spread is 1.1×–1.6×, so most of the gap lies between stacks that do not share them; we do not decompose the rest.

Second, on the easiest dataset the effect is energetic and not qualitative — all 7 stacks converge to within 0.5 percentage points on Fashion-MNIST — but on the harder datasets they differ by up to 6.5 points per cell, and energy must be read alongside accuracy.

Third, and unexpectedly, the apparatus decides part of the answer, though not the energy: a widely used estimator agrees with the counters to 0.3 % on energy but pads the duration it reports in 75 % of blocks, understating derived power by up to 13.0×. On measured durations energy and time rank the configurations at $\rho = 0.96$ in training: on this platform and workload faster *is* greener, and the estimator's durations weaken that relation without reversing it.

Fourth, in a contrast cell at 224×224 that loads the accelerator for most stacks, the spread within the LibTorch lineage compresses to 1.3×–2.8× while across all 7 it widens to 15.3×–19.1×, carried by the one stack without a cuDNN path. The effect survives a heavier per-sample workload; its size and its cheapest stack do not carry over unchanged.

A cross-ecosystem energy study therefore needs a written specification, a mechanism that enforces it and a second instrument, for the same reason a performance benchmark needs a stated warm-up policy. Ours — the specification, the measurement contract, the 110-check conformance checker and the raw dual-instrument records — are in the replication package, and every figure in this paper is generated from them except where stated; Online Resource 1 lists the exceptions (Sects. C.5 and D.3).

11 Future Work

The workload should extend to transformer and multimodal architectures, whose kernels are attention and matrix multiplication rather than convolution, and to a sweep of resolution and batch size; a cuDNN-off measurement at 224×224, ideally in nd4j itself, would show how much of Java's distance at saturation its missing backend accounts for. Server-class accelerators, TPUs and edge devices, which saturate at different points from a desktop card, would separate the stacks from the host. A white-box layer would localise the 1.1×–1.6× that survives a fixed model, data and backend build: loader

scheduling, host-to-device transfer, launch overhead and allocation on one side, kernel and algorithm selection and precision on the other. The reported-window padding invites replication across estimators, sampling intervals and tracking modes, and a wall-meter reference would relate our boundary to whole-system energy.

12 Carbon Footprint of This Study

The campaign consumed 40.4 MJ (11.2 kWh) at the accelerator and CPU package inside its measured blocks, over 210 runs; at the grid carbon intensity CodeCarbon assigns to this location, a few kilograms of CO_2e. The saturation cell, reported separately because nothing here pools it with the campaign, consumed 68.9 MJ (19.1 kWh) over 70 runs on the same definition. Both figures exclude development, failed runs and the audited and superseded campaigns (the first on other hardware and at another boundary), which would make them several times larger, and the draw between blocks: a further 2.4 MJ for the campaign (Section 6.5), unmeasured for the cell. They are the quantity the comparisons are made on, not a campaign-level total.

Declarations

Competing interests. The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Ethics approval and consent to participate. Not applicable.

Consent for publication. Not applicable.

Data availability. All data and code underlying this study are public in the replication package at https://github.com/Pampaj7/DeepGreen; an archived copy with a DOI will accompany the published version. The package contains the implementations for all 7 ecosystems, the shared measurement bridge, the experiment specification and its conformance checker, the campaign driver, the per-block dual-instrument records for all 210 runs and 12 600 blocks, and the analysis pipeline that turns those records into the tables, figures and quoted numbers in this manuscript. The saturation cell's records — the same per-block dual-instrument readings, per-epoch metrics, run manifests and data fingerprints for its 70 runs, with the excerpt of the utilisation record that covers them — are in a directory of the package of their own, built by the same pipeline and never merged with the campaign's. A single command regenerates the manuscript from the raw records, so every number here can be traced to the measurement it came from, except where stated; Online Resource 1 lists the exceptions (Sects. C.5 and D.3).

Code availability. The implementations for all 7 ecosystems, the shared measurement bridge, the experiment specification and its conformance checker, the campaign driver and the analysis pipeline are part of the public replication package described under Data availability, https://github.com/Pampaj7/DeepGreen.

Author contributions. **Leonardo Pampaloni:** Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Data curation, Writing — original draft, Visualization. **Marco Pagliocca:** Methodology, Software, Validation, Investigation, Data curation, Writing — original draft, Visualization. **Enrico Vicario:** Resources, Supervision, Funding acquisition, Writing — review & editing. **Roberto Verdecchia:** Conceptualization, Methodology, Validation, Writing — review & editing, Supervision, Project administration.

Use of generative AI and large language models. During the preparation of this work the authors used generative AI assistants (ChatGPT and Claude) in three distinct capacities, which we separate because they carry different weight.

For *writing*, the tools were used to improve the readability of the manuscript text. For *software engineering*, they were used to repair and refactor the experimental implementations, to write the shared measurement bridge, the conformance checker and the analysis pipeline, and to diagnose defects in the ecosystems' data loaders and build configurations. For *analysis*, they were used to write the scripts that produce the tables and figures; the scripts, not the assistants, produce the numbers, and the numbers are injected into this manuscript from those scripts rather than transcribed.

No experimental measurement was produced, adjusted or selected by an AI assistant. The authors reviewed and edited all generated content and code, verified the findings against the raw data, and take full responsibility for the content of the article.

References

Aashish KC, et al (2026) Towards eco friendly cybersecurity: machine learning based anomaly detection with carbon and energy metrics. arXiv preprint arXiv:2601.00893

Alizadeh N, Castor F (2024) Green AI: A preliminary empirical study on energy consumption in DL models across different runtime infrastructures. In: Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering — Software Engineering for AI (CAIN), pp 134–139, https://doi.org/10.1145/3644815.3644967

Almog T (2025) An analysis of optimizer choice on energy efficiency and performance in neural network training. arXiv preprint arXiv:2509.13516, https://doi.org/10.48550/arXiv.2509.13516

Ampatzoglou A, Bibi S, Avgeriou P, et al (2019) Identifying, categorizing and mitigating threats to validity in software engineering secondary studies. Information and Software Technology 106:201–230. https://doi.org/10.1016/j.infsof.2018.10.006

Aquino-Brítez S, García-Sánchez P, Ortiz A, et al (2025) Towards an energy consumption index for deep learning models: A comparative analysis of architectures, GPUs, and measurement tools. Sensors 25(3):846. https://doi.org/10.3390/s25030846

Banbury C, Reddi VJ, Torelli P, et al (2021) MLPerf Tiny benchmark. In: Proceedings of the NeurIPS 2021 Track on Datasets and Benchmarks, arXiv:2106.07597

Basili VR, Caldiera G, Rombach HD (1994) The goal question metric approach. In: Encyclopedia of Software Engineering, vol 1. John Wiley & Sons, pp 528–532

Bender EM, Gebru T, McMillan-Major A, et al (2021) On the dangers of stochastic parrots: Can language models be too big? In: Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp 610–623, https://doi.org/10.1145/3442188.3445922

Bouza L, Bugeau A, Lannelongue L (2023) How to estimate carbon footprint when training deep learning models? A guide and review. Environmental Research Communications 5(11):115014. https://doi.org/10.1088/2515-7620/acf81b

Campbell DT, Stanley JC (1963) Experimental and Quasi-Experimental Designs for Research. Houghton Mifflin, Boston

Chandra TB, Verma P, Dwivedi AK (2019) Impact of programming languages on energy consumption for sorting algorithms. In: Software Engineering, Advances in Intelligent Systems and Computing, vol 731. Springer, Singapore, https://doi.org/10.1007/978-981-10-8848-3_9

Chattaraj R, Chimalakonda S, Sharma VS, et al (2025) Who wins the race? (R vs Python) — an exploratory study on energy consumption of machine learning algorithms. arXiv preprint arXiv:2508.17344

Cook TD, Campbell DT (1979) Quasi-Experimentation: Design and Analysis Issues for Field Settings. Houghton Mifflin, Boston

Courty B, Schmidt V, Luccioni S, et al (2026) `mlco2/codecarbon`: v3.3.0. https://doi.org/10.5281/zenodo.4658424, URL https://github.com/mlco2/codecarbon

Cruz L, Fernandes JP, Kirkeby MH, et al (2025) Greening AI-enabled systems with software engineering: A research agenda for environmentally sustainable AI practices. ACM SIGSOFT Software Engineering Notes https://doi.org/10.1145/3743095.3743099, arXiv:2506.01774

Dodge J, Prewitt T, Tachet des Combes R, et al (2022) Measuring the carbon intensity of AI in cloud instances. In: Proceedings of the 2022 ACM Conference on Fairness, Accountability, and Transparency (FAccT), pp 1877–1894, https://doi.org/10.1145/3531146.3533234

Feldt R, Magazinius A (2010) Validity threats in empirical software engineering research – an initial survey. In: Proceedings of the 22nd International Conference on Software Engineering & Knowledge Engineering (SEKE), pp 374–379

Fischer R (2026) Ground-truthing AI energy consumption: validating CodeCarbon against external measurements. it - Information Technology 67(4-6):155–163. https://doi.org/10.1515/itit-2025-0031

Georgiou S, Kechagia M, Spinellis D (2017) Analyzing programming languages' energy consumption: An empirical study. In: Proceedings of the 21st Pan-Hellenic Conference on Informatics (PCI), https://doi.org/10.1145/3139367.3139418

Georgiou S, Kechagia M, Sharma T, et al (2022) Green AI: Do deep learning frameworks have different costs? In: Proceedings of the 44th International Conference on Software Engineering (ICSE), pp 1082–1094, https://doi.org/10.1145/3510003.3510221

Gordillo A, Calero C, Moraga MÁ, et al (2024) Programming languages ranking based on energy measurements. Software Quality Journal 32(4):1539–1580. https://doi.org/10.1007/s11219-024-09690-4

He K, Zhang X, Ren S, et al (2016) Deep residual learning for image recognition. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), pp 770–778, https://doi.org/10.1109/CVPR.2016.90, arXiv:1512.03385

Henderson P, Hu J, Romoff J, et al (2020) Towards the systematic reporting of the energy and carbon footprints of machine learning. Journal of Machine Learning Research 21(248):1–43

Jay M, Ostapenco V, Lefèvre L, et al (2023) An experimental comparison of software-based power meters: Focus on CPU and GPU. In: Proceedings of the 23rd IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGrid), pp 106–118, https://doi.org/10.1109/CCGrid57682.2023.00020

Kaack LH, Donti PL, Strubell E, et al (2022) Aligning artificial intelligence with climate change mitigation. Nature Climate Change 12(6):518–527. https://doi.org/10.1038/s41558-022-01377-7

van Kempen N, Kwon HJ, Nguyen DT, et al (2025) It's not easy being green: On the energy efficiency of programming languages. In: Proceedings of the 40th IEEE/ACM International Conference on Automated Software Engineering (ASE), https://doi.org/10.1109/ASE63991.2025.00131, arXiv:2410.05460

Khan KN, Hirki M, Niemi T, et al (2018) RAPL in action: Experiences in using RAPL for power measurements. ACM Transactions on Modeling and Performance Evaluation of Computing Systems 3(2):9:1–9:26. https://doi.org/10.1145/3177754

Krizhevsky A (2009) Learning multiple layers of features from tiny images. Tech. rep., University of Toronto

Lacoste A, Luccioni A, Schmidt V, et al (2019) Quantifying the carbon emissions of machine learning. arXiv preprint arXiv:1910.09700

Le Y, Yang XS (2015) Tiny ImageNet visual recognition challenge. Tech. Rep. CS231N course report, Stanford University, URL http://cs231n.stanford.edu/reports/2015/

pdfs/yle_project.pdf

Li D, Chen X, Becchi M, et al (2016) Evaluating the energy efficiency of deep convolutional neural networks on CPUs and GPUs. In: Proceedings of the 2016 IEEE International Conferences on Big Data and Cloud Computing (BDCloud), Social Computing and Networking (SocialCom), Sustainable Computing and Communications (SustainCom), pp 477–484, https://doi.org/10.1109/BDCloud-SocialCom-SustainCom.2016.76

Li H, Rajbahadur GK, Bezemer CP (2024) Studying the impact of TensorFlow and PyTorch bindings on machine learning software quality. ACM Transactions on Software Engineering and Methodology 34(1):1–31. https://doi.org/10.1145/3678168

Luccioni AS, Hernandez-Garcia A (2023) Counting carbon: A survey of factors influencing the emissions of machine learning. arXiv preprint arXiv:2302.08476

Luccioni AS, Viguier S, Ligozat AL (2023) Estimating the carbon footprint of BLOOM, a 176B parameter language model. Journal of Machine Learning Research 24(253):1–15. ArXiv:2211.02001

Luccioni AS, Jernite Y, Strubell E (2024) Power hungry processing: Watts driving the cost of AI deployment? In: Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24). Association for Computing Machinery, Rio de Janeiro, Brazil, pp 85–99, https://doi.org/10.1145/3630106.3658542

Mahadevappa S, Figueira S (2021) Energy-efficient programming languages for mobile applications. In: 2021 IEEE Global Humanitarian Technology Conference (GHTC), https://doi.org/10.1109/GHTC53159.2021.9612479

Maleki S, Fu C, Banotra A, et al (2017) Understanding the impact of object oriented programming and design patterns on energy efficiency. In: 2017 Eighth International Green and Sustainable Computing Conference (IGSC), pp 1–6, https://doi.org/10.1109/IGCC.2017.8323605

Marini N, Pampaloni L, Di Martino F, et al (2025) Green AI: Which programming language consumes the most? In: 2025 IEEE/ACM International Workshop on Green and Sustainable Software (GREENS), arXiv:2501.14776

Mehlin V, Schacht S, Lanquillon C (2023) Towards energy-efficient deep learning: An overview of energy-efficient approaches along the deep learning lifecycle. arXiv preprint arXiv:2303.01980

Nanz S, Furia CA (2015) A comparative study of programming languages in Rosetta Code. In: Proceedings of the 37th International Conference on Software Engineering (ICSE), pp 778–788, arXiv:1409.0252

Patterson D, Gonzalez J, Le Q, et al (2021) Carbon emissions and large neural network training. arXiv preprint arXiv:2104.10350

Patterson D, Gonzalez J, Hölzle U, et al (2022) The carbon footprint of machine learning training will plateau, then shrink. Computer 55(7):18–28. https://doi.org/10.1109/MC.2022.3148714

Paula E, Soni J, Upadhyay H, et al (2025) Comparative analysis of model compression techniques for achieving carbon efficient AI. Scientific Reports 15:23461. https://doi.org/10.1038/s41598-025-07821-w

Pereira R, Couto M, Ribeiro F, et al (2017) Energy efficiency across programming languages: how do energy, time, and memory relate? In: Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE), pp 256–267, https://doi.org/10.1145/3136014.3136031

Pereira R, Couto M, Ribeiro F, et al (2021) Ranking programming languages by energy efficiency. Science of Computer Programming 205:102609. https://doi.org/10.1016/j.scico.2021.102609

Rua R, Saraiva J (2024) A large-scale empirical study on mobile performance: energy, run-time and memory. Empirical Software Engineering 29(1):31. https://doi.org/10.1007/s10664-023-10391-y

Schwartz R, Dodge J, Smith NA, et al (2020) Green AI. Communications of the ACM 63(12):54–63. https://doi.org/10.1145/3381831

Simonyan K, Zisserman A (2015) Very deep convolutional networks for large-scale image recognition. In: 3rd International Conference on Learning Representations (ICLR), arXiv:1409.1556

Sjøberg DIK, Bergersen GR (2023) Construct validity in software engineering. IEEE Transactions on Software Engineering 49(3):1374–1396. https://doi.org/10.1109/TSE.2022.3176725

Strubell E, Ganesh A, McCallum A (2019) Energy and policy considerations for deep learning in NLP. In: Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), pp 3645–3650, https://doi.org/10.18653/v1/P19-1355

Tripp CE, Perr-Sauer J, Gafur J, et al (2024) Measuring the energy consumption and efficiency of deep neural networks: An empirical analysis and design recommendations. arXiv preprint arXiv:2403.08151

Verdecchia R, Engström E, Lago P, et al (2023a) Threats to validity in software engineering research: A critical reflection. Information and Software Technology 164:107329. https://doi.org/10.1016/j.infsof.2023.107329

Verdecchia R, Sallou J, Cruz L (2023b) A systematic review of Green AI. WIREs Data Mining and Knowledge Discovery 13(4):e1507. https://doi.org/10.1002/widm.1507

Weber M, Kaltenecker C, Sattler F, et al (2023) Twins or false friends? A study on energy consumption and performance of configurable software. In: Proceedings of the 45th International Conference on Software Engineering (ICSE), pp 2098–2110, https://doi.org/10.1109/ICSE48619.2023.00177

Xiao H, Rasul K, Vollgraf R (2017) Fashion-MNIST: a novel image dataset for benchmarking machine learning algorithms. arXiv preprint arXiv:1708.07747

Xu J, Zhou W, Fu Z, et al (2021) A survey on green deep learning. arXiv preprint arXiv:2111.05193

Yarally T, Cruz L, Feitosa D, et al (2023) Uncovering energy-efficient practices in deep learning training: Preliminary steps towards green AI. In: 2023 IEEE/ACM 2nd International Conference on AI Engineering – Software Engineering for AI (CAIN), pp 25–36, arXiv:2303.13972